

FINAL PROJECT GROUP REPORT

Application of Regression Models to Improve Sales Performance

An End-to-End Analysis using Python, SQL, Metabase & Power BI

Group Name / No.	Group 2 - Regression
Name	Dzulfi Khoiriyah Azzahra
Training Program	Data Analyst & Data Science
Training Center	BLKPP DIY
Instructor	Denta Winardi & Kuncoro
Submission Date	22 June 2026

*This report is submitted as a requirement for completion of the
Data Analyst & Data Science mini-project module.*

Table of Contents

1. Executive Summary.....	4
2. Introduction	4
2.1 Background.....	4
2.2 Problem Statement	4
2.3 Objectives.....	4
3. Dataset Description.....	5
3.1 Source	5
3.2 Variables / Columns.....	5
3.3 Data Dictionary	5
4. Methodology.....	6
4.1 Python	6
4.2 SQL.....	6
4.3 Metabase.....	6
4.4 Power BI.....	6
4.5 Streamlit	6
5. Data Cleaning & Preprocessing.....	7
5.1 Loading & Inspecting the Data	8
5.2 Data Cleaning.....	8
5.3 Feature Engineering	8
5.4 Correlation Analysis.....	9
5.5 Data Preparation for Modeling	9
5.6 Model Development.....	10
5.7 Model Evaluation.....	11
6. SQL Analysis	12
6.1 Yearly Trend.....	12
6.2 Monthly Trend.....	12
6.3 Sales Performance by Day of Week and Promotion	13
6.4 Weekend vs. Weekday Sales Comparison.....	14
6.5 Sales Comparison During National Online Shopping Day (Harbolnas).....	14
6.6 Sales Comparison During Promotional Periods.....	14
6.7 Sales Comparison During School Holidays	15
6.8 Sales Comparison During Payday Periods	15
6.9 Store Performance Analysis	15
7. Data Visualization.....	17
7.1 Metabase — Exploratory Dashboard	17
7.2 Power BI — Executive Dashboard	18
7.3 Supporting Chart — Revenue by Category sales.....	19

7.4 Streamlit— Sales Prediction Application.....	19
8. Key Findings & Insights	20
9. Conclusion & Recommendations	21
9.1 Conclusion	21
9.2 Recommendations.....	22
A. Improve Customer Acquisition and Retention	22
B. Optimize Promotions During High-Potential Periods	22
C. Evaluate Underperforming Stores	22
D. Use Sales Patterns for Business Planning	22
9.3 Limitations & Future Work	22
10. Appendix	22
Appendix A — Full Python Script For Data Cleaning and Model Deployment	22
Appendix B — Full SQL Script for Finding Insights	27
Appendix C — Python Script for Deployment	29
Appendix D — Team Contribution Table.....	35

1. Executive Summary

This report analyzes 610,234 daily sales records from 1,115 retail stores for the period January 2014–July 2015. The goal was to identify the factors influencing sales, evaluate store performance, and develop a regression model to support sales forecasting and business decision-making.

The team used Python (pandas) for data cleaning and feature engineering, SQL for sales aggregation, and Metabase and Power BI for interactive dashboards. Five regression models were evaluated, with XGBoost achieving the best performance with an R^2 of 0.80 and MAPE of 21%. Key findings show that: (1) sales are strongly associated with customer traffic; (2) promotional periods generated €1.9 billion in sales, compared with €1.7 billion during non-promotional periods; and (3) Stores 262, 562, and 1,114 were identified as underperforming in both average sales and customer traffic.

We recommend strengthening customer acquisition and loyalty programs, targeting promotions based on customer traffic and seasonal patterns, and conducting operational evaluations of underperforming stores. The XGBoost forecasting model can also be used to support promotion planning, store operations, and future sales targets.

2. Introduction

2.1 Background

This dataset contains daily sales data from various stores, including information on sales, the number of customers, promotions, holidays, and time-related attributes such as day, month, and quarter. In addition to operational data, the dataset also includes data engineering features, such as pay periods, weekends, and promotional events, which can be used to identify customer behavior patterns and sales trends. This dataset is used to analyze factors that influence sales and to build regression models to predict sales and support more effective business decision-making.

2.2 Problem Statement

The company faces uncertainty in forecasting daily sales, while some stores are still reporting sales below target and sales growth that tends to be stagnant. These conditions make it difficult for the company to devise effective promotional and operational strategies to boost sales.

2.3 Objectives

- Analyze the impact of promotions, customer count, and time-related factors on sales.
- Identify stores with low sales performance.
- Build a regression model to predict daily sales.

3. Dataset Description

3.1 Source

This dataset consists of fictitious sales data provided by the instructor in the form of a single Excel file, namely regression.xlsx. The file contains 610,234 rows (one row per order) and 21 columns, covering all orders completed between January 1, 2014, and July 31, 2015. Actual customer names are not included; customer identities are represented only by anonymized IDs.

3.2 Variables / Columns

This dataset includes information on sales, the number of customers, promotions, holidays, and time-related attributes such as day, month, and quarter. In addition to operational data, the dataset also includes features derived from data engineering, such as pay periods, weekends, and promotional events. The table below provides a description of each column.

3.3 Data Dictionary

Column	Data Type	Description	Example
Store	Integer	Unique identifier for each store	909
DayOfWeek	Integer	Day of the week (1–7, depending on dataset definition)	1
Date	Date	Transaction or record date	22/06/2015
Sales	Integer	Total sales on that specific day	41551
Customers_id	Integer	Number of unique customers or aggregated customer identifier	1721
Open	Integer	Store status (1 = open, 0 = closed)	1
Promo	Integer	Indicates whether a promotion is active (1 = yes, 0 = no)	0
StateHoliday	Object	Type of state holiday (e.g., a, b, c, or 0 if none)	0
SchoolHoliday	Integer	Indicates school holiday (1 = yes, 0 = no)	0
year	Float	Year of the transaction	2015
month	Float	Month of the transaction (1–12)	6
day_of_month	Float	Day of the month (1–31)	22
week_of_year	Float	Week number within the year (1–52/53)	26
quarter	Float	Quarter of the year (1–4)	2
is_weekend	Float	Indicates whether the day is a weekend (1 = yes, 0 = no)	0
is_payday_period	Float	Indicates payday period (e.g., beginning or end of month)	0
event_harbolnas	Float	Indicates Harbolnas (National Online Shopping Day) event period	0
kategori_sales	Float	Sales category (e.g., low, medium, high)	Very High
store_avg_customer	Float	Average number of customers per store	813,6
customer_ratio	Float	Ratio of customers relative to a baseline (e.g., per store/day)	8720,4
store_avg_sales	Float	Average sales per store	2,1

Table 3.1 — Data dictionary for regression.xlsx

4. Methodology

Our data pipeline consists of five stages. First, raw Excel data is cleaned and prepared using Python. Second, the cleaned data is loaded into a relational database and analyzed using SQL. Third, the database is connected to Metabase to create fast, shareable exploratory dashboards. Fourth, a refined executive dashboard is created in Power BI for executive reporting. Finally, the sales prediction model is deployed using Streamlit, allowing users to generate real-time sales forecasts through an interactive web-based interface.

4.1 Python

Role: Data cleaning, preprocessing, and exploratory analysis. We used the pandas and NumPy libraries to load Excel, handle missing values, fix data types, remove duplicates, and engineer new columns (such as order month and customer type). Python was chosen because it handles messy raw data flexibly and reproducibly through code.

4.2 SQL

Role: Structured grouping and query processing. After loading the cleaned data into the SQLite database, we wrote SQL queries to answer specific business questions—revenue by category, monthly trends, top customers, and repeat purchase rates. SQL is well-suited for efficiently grouping, filtering, and joining large tables.

4.3 Metabase

Role: Rapid, browser-based exploratory dashboards. Metabase connected directly to our database, letting non-technical stakeholders run questions and view charts without writing code. We used it for quick visual checks and an internal exploratory dashboard.

4.4 Power BI

Role: Final presentation dashboard. Power BI was used to build a polished, interactive executive dashboard with slicers (filters) for region and time period. It produces a professional layout suitable for presenting findings to management.

4.5 Streamlit

Role: Streamlit was used as a deployment layer to transform the trained machine learning model into an interactive web application. It enables users to input relevant variables and obtain real-time predictions through a browser-based. The tool integrates model loading and inference in a simple Python environment, making the analytical results more accessible for end-users without requiring technical expertise.

5. Data Cleaning & Preprocessing

The first stage performed after understanding the dataset was data preparation, which consisted of data inspection, feature engineering, encoding, feature selection, and data scaling. The steps carried out are as follows:

1. Import the Excel dataset into Python (Google Colab) and examine the dataset structure, data types, descriptive statistics, missing values, and duplicate records.
2. Create new date-related features from the **Date** column, including **year**, **month**, **day_of_month**, **week_of_year**, and **quarter**.
3. Create a new feature called **is_weekend** to identify whether a transaction occurred on a weekend.
4. Create a new feature called **is_payday_period** to identify transactions occurring during the payday period (from the 25th day of the month onward).
5. Create a new feature called **event_harbolnas** to identify promotional dates where the day and month values are the same (e.g., 1/1, 2/2, 3/3, and so on).
6. Generate customer and store performance metrics by creating the following variables:
 - **store_avg_customer**, representing the average number of customers for each store.
 - **store_avg_sales**, representing the average sales value for each store.
 - **customer_ratio**, representing the ratio between the number of customers and the average number of customers in each store.
7. Create a new categorical feature called **kategori_sales** by dividing the Sales variable into four quartile-based categories: Low, Medium, High, and Very High.
8. Perform outlier detection on all numerical variables using the **Interquartile Range (IQR)** method to identify extreme observations.
9. Encode categorical variables by applying:
 - **Ordinal Encoding** to the **kategori_sales** feature.
 - **One-Hot Encoding** to the **StateHoliday** feature.
10. Perform feature alignment to ensure that the training and testing datasets contain the same set of variables.
11. Conduct feature selection by removing variables considered less relevant to the prediction objective, including **Customers_id**, **year**, **Open**, **kategori_sales**, **quarter**, **event_harbolnas**, **store_avg_sales**, and **customer_ratio**.
12. Apply **RobustScaler** to standardize the selected features and reduce the influence of outliers before model training.
13. Use the processed dataset to build and evaluate several regression models, including **Linear Regression**, **Ridge Regression**, **Lasso Regression**, **AdaBoost Regression**, and **XGBoost Regression**.

5.1 Loading & Inspecting the Data

Python — load the dataset and review its structure

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_excel('regresi.xlsx')

print(df.shape)          # (12614, 12) before cleaning
print(df.info())         # column types and non-null counts
print(df.isnull().sum()) # missing values per column
print(df.duplicated().sum()) # number of duplicate rows
```

5.2 Data Cleaning

Python — Check For Duplicate Data and Missing Values

```
# Check duplicate records
df.duplicated().sum()

# Check Missing values
df.isnull().sum()
```

5.3 Feature Engineering

Python — create helper columns for analysis

```
# Extract year, month, day, week, and quarter from the date column
df['year'] = df['Date'].dt.year
df['month'] = df['Date'].dt.month
df['day_of_month'] = df['Date'].dt.day
df['week_of_year'] = df['Date'].dt.isocalendar().week.astype(int)
df['quarter'] = df['Date'].dt.quarter

# Create a binary feature indicating whether the date falls on a weekend
df['is_weekend'] = df['Date'].dt.dayofweek.isin([5, 6]).astype(int)

# Create a payday-period indicator (from the 25th until month-end and the 2nd day)
df['is_payday_period'] = np.where(
    (df['day_of_month'] >= 25) | (df['day_of_month'] == 2), 1, 0)

# Create an indicator for Harbolnas-style promotional events (e.g., 11/11, 12/12)
df['event_harbolnas'] = np.where(
    df['day_of_month'] == df['month'], 1, 0)

# Calculate customer traffic relative to the store's average customer count
df['customer_ratio'] = (
    df['Customers_id'] /
    df.groupby('Store')['Customers_id'].transform('mean'))

# Categorize sales into four quartile-based levels
df['kategori_sales'] = pd.qcut(
    df['Sales'],
    q=4,
    labels=['Low', 'Medium', 'High', 'Very High'])

# Calculate the average number of customers for each store
```



```
df['store_avg_customer'] = (
    df.groupby('Store')['Customers_id']
    .transform('mean'))

# Calculate the average sales for each store
df['store_avg_sales'] = (
    df.groupby('Store')['Sales']
    .transform('mean'))

# Export the final dataset to an Excel file
df.to_excel('regresi_final.xlsx', index=False)
```

5.4 Correlation Analysis

Python — Analyzing relationships among numerical variables

```
# Check duplicate records
corr_matrix = df.corr(numeric_only=True)

plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.show()
```

5.5 Data Preparation for Modeling

Python — selecting features and target variables

```
# Split Data
X = df.drop(['Sales', 'Date'], axis=1)
y = df['Sales']

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42 )

# Ordinal Encoding
mapping = {
    'Low': 1,
    'Medium': 2,
    'High': 3,
    'Very High': 4 }
X_train['kategori_sales'] = X_train['kategori_sales'].map(mapping)
X_test['kategori_sales'] = X_test['kategori_sales'].map(mapping)

# One Hot Encoding
X_train = pd.get_dummies(X_train, columns=['StateHoliday'], drop_first=True, dtype=int)
X_test = pd.get_dummies(X_test, columns=['StateHoliday'], drop_first=True, dtype=int)

# Align columns after one-hot encoding
train_cols = X_train.columns
test_cols = X_test.columns

missing_in_test = set(train_cols) - set(test_cols)
for c in missing_in_test:
    X_test[c] = 0

missing_in_train = set(test_cols) - set(train_cols)
for c in missing_in_train:
    X_train[c] = 0
```

```
X_test = X_test[train_cols]

# Feature Selection
drop_cols = [
    'Customers_id',
    'year',
    'Open',
    'kategori_sales',
    'quarter',
    'event_harbolnas',
    'store_avg_sales',
    'customer_ratio' ]
X_train = X_train.drop(columns=drop_cols, errors='ignore') X_test =
X_test.drop(columns=drop_cols, errors='ignore')
```

5.6 Model Development

Python — Training the regression model

```
# Linear Regression
lr = LinearRegression(
    fit_intercept=True,
    copy_X=True,
    positive=False )
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

# Ridge Regression
ridge = Ridge(
    alpha=1.0,
    fit_intercept=True,
    solver='auto',
    max_iter=None, tol=0.0001,
    random_state=None )
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

# Lasso Regression
lasso = Lasso(
    alpha=1.0,
    fit_intercept=True,
    max_iter=1000, tol=0.0001,
    selection='cyclic',
    random_state=None )
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

#SVR (Support Vector Regression)
svr = SVR(kernel='rbf')
svr.fit(X_train, y_train)
y_pred_svr= svr.predict(X_test)

# AdaBoost Regression
ada = AdaBoostRegressor(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)
y_pred_ada = ada.predict(X_test)

#XGBoost Regression
```

```
xgb = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
    enable_categorical=True)
xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)
```

5.7 Model Evaluation

Python — Evaluating prediction performance

```
# Check duplicate records
from sklearn.metrics import (
    mean_squared_error,
    mean_absolute_error,
    mean_absolute_percentage_error,
    r2_score )
result = []
mask = y_test > 0

models = {
    'Linear Regression': y_pred_lr,
    'Ridge Regression': y_pred_ridge,
    'Lasso Regression': y_pred_lasso,
    'SVR': y_pred_svr,
    'Adaboost Regression': y_pred_ada,
    'XGBoost Regression': y_pred_xgb }
for name, y_pred in models.items():
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mape = mean_absolute_percentage_error(y_test[mask], y_pred[mask]) * 100
    r2 = r2_score(y_test, y_pred) result.append([name, mae, rmse, mape, r2])
comparison_df = pd.DataFrame( result, columns=['Model', 'MAE', 'RMSE', 'MAPE (%)', 'R²'])
comparison_df = comparison_df.round(2)
comparison_df

# Feature Importance
importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': xgb.feature_importances_ })
importance = importance.sort_values(
    'Importance',
    ascending=False)
importance.head(10)
```

Result: After data cleaning and feature engineering, the final dataset contained **610.234 rows and 21 columns**, including newly created temporal, promotional, customer behavior, and store-level features. Correlation analysis indicated that customer traffic was the strongest predictor of sales. The dataset was then split into training and testing sets and modeled using **XGBoost Regression**. The model achieved an **R² score of 80%**, demonstrating strong predictive performance in estimating future sales. Feature importance analysis revealed that **Customers_id, store_avg_sales, store_avg_customer, and promotional indicators** were among the most influential variables affecting sales predictions.

6. SQL Analysis

6.1 Yearly Trend

Purpose: Analyze annual sales performance and the growth of unique customers to identify business growth trends, customer base development, and overall performance changes over time.

A. Yearly Sales Trend

```
select "year" as tahun,
       count(distinct "Customers_id") as pelanggan,
       sum("Sales") as penjualan
from   regresi
group  by "year";
```

B. Yearly Customer Trend

```
select "year" as tahun,
       count(distinct "Customers_id") as pelanggan,
       sum("Sales") as penjualan
from   regresi
group  by "year";
```

Finding: Sales have shown a year-over-year downward trend, in line with the decline in the number of customers. This indicates that the decline in customers is one of the main factors affecting the company's sales performance.

6.2 Monthly Trend

Purpose: Identify seasonal patterns and monthly fluctuations in sales and customer traffic to support business planning and determine peak activity periods.

A. Monthly Sales Trend

```
SELECT DATE_TRUNC('month', "Date") AS bulan,
       SUM("Sales") AS total_sales
FROM   regresi
GROUP  BY DATE_TRUNC('month', "Date")
ORDER  BY bulan;
```

B. Monthly Customer Trend

```
SELECT
    DATE_TRUNC('month', "Date") AS bulan,
    count(distinct "Customers_id") as pelanggan
FROM   regresi
GROUP  BY DATE_TRUNC('month', "Date")
ORDER  BY bulan;
```

Finding: Sales have shown a fluctuating pattern throughout the year, with some months experiencing significant increases and others experiencing declines. This indicates that sales are influenced by seasonal factors.

6.3 Sales Performance by Day of Week and Promotion

Purpose: Evaluate the effectiveness of promotional campaigns across different days of the week.

A. Average Sales by Day of Week and Promotion

```
SELECT
  CASE
    WHEN "DayOfWeek" = 1 THEN 'Monday'
    WHEN "DayOfWeek" = 2 THEN 'Tuesday'
    WHEN "DayOfWeek" = 3 THEN 'Wednesday'
    WHEN "DayOfWeek" = 4 THEN 'Thursday'
    WHEN "DayOfWeek" = 5 THEN 'Friday'
    WHEN "DayOfWeek" = 6 THEN 'Saturday'
    WHEN "DayOfWeek" = 7 THEN 'Sunday'
  END AS day_name,
  AVG(CASE WHEN "Promo" = 0 THEN "Sales" END) AS no_promo,
  AVG(CASE WHEN "Promo" = 1 THEN "Sales" END) AS promo
FROM regresi
WHERE "Open" = 1
GROUP BY "DayOfWeek"
ORDER BY "DayOfWeek";
```

B. Average Customers by Day of Week

```
SELECT
  CASE
    WHEN "DayOfWeek" = 1 THEN 'Monday'
    WHEN "DayOfWeek" = 2 THEN 'Tuesday'
    WHEN "DayOfWeek" = 3 THEN 'Wednesday'
    WHEN "DayOfWeek" = 4 THEN 'Thursday'
    WHEN "DayOfWeek" = 5 THEN 'Friday'
    WHEN "DayOfWeek" = 6 THEN 'Saturday'
    WHEN "DayOfWeek" = 7 THEN 'Sunday'
  END AS day_name,
  AVG("Customers_id") AS avg_customers
FROM regresi
WHERE "Open" = 1
GROUP BY "DayOfWeek"
ORDER BY "DayOfWeek";
```

C. Average Sales by Day of Week

```
SELECT
  CASE
    WHEN "DayOfWeek" = 1 THEN 'Monday'
    WHEN "DayOfWeek" = 2 THEN 'Tuesday'
    WHEN "DayOfWeek" = 3 THEN 'Wednesday'
    WHEN "DayOfWeek" = 4 THEN 'Thursday'
    WHEN "DayOfWeek" = 5 THEN 'Friday'
    WHEN "DayOfWeek" = 6 THEN 'Saturday'
    WHEN "DayOfWeek" = 7 THEN 'Sunday'
  END AS day_name,
  AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "DayOfWeek"
ORDER BY avg_sales DESC;
```

Finding: The store appears to rely heavily on weekday promotions to boost sales, but fails to maximize the value of purchases from the large crowds on weekends.

6.4 Weekend vs. Weekday Sales Comparison

Purpose: Compare sales and customer behavior between weekdays and weekends.

```
SELECT
    CASE
        WHEN is_weekend = 1 THEN 'Weekend'
        WHEN is_weekend = 0 THEN 'Weekday'
    END AS period_type,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY is_weekend;
```

Finding: Weekdays generate significantly higher average sales than weekends, indicating that most customer transactions occur during regular working days.

6.5 Sales Comparison During National Online Shopping Day (Harbolnas)

Purpose: Measure the impact of Harbolnas events on sales performance and customer engagement.

```
SELECT
    CASE
        WHEN event_harbolnas = 1 THEN 'Harbolnas Event'
        WHEN event_harbolnas = 0 THEN 'Non-Harbolnas'
    END AS event_status,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY event_harbolnas;
```

Finding: Average sales during Harbolnas events are higher than on regular days, suggesting that the event successfully boosts customer purchases.

6.6 Sales Comparison During Promotional Periods

Purpose: Assess the effectiveness of promotional campaigns in increasing sales and customer traffic.

```
SELECT
    CASE
        WHEN "Promo" = 1 THEN 'Promotion'
        WHEN "Promo" = 0 THEN 'No Promotion'
    END AS promotion_status,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY "Promo";
```

Finding: Total sales during promotional periods (€1.9 billion) exceeded sales during non-promotional periods (€1.7 billion). This indicates that, overall, the promotional strategy implemented was able to attract more transactions or increase the value of customers' shopping carts.

6.7 Sales Comparison During School Holidays

Purpose: Analyze the impact of school holiday periods on sales performance and customer traffic.

```
SELECT
    CASE
        WHEN "SchoolHoliday" = 1 THEN 'School Holiday'
        WHEN "SchoolHoliday" = 0 THEN 'Non-School Holiday'
    END AS holiday_status,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY "SchoolHoliday";
```

Finding: Average sales are noticeably higher during school holidays, suggesting that customers tend to spend more during these periods.

6.8 Sales Comparison During Payday Periods

Purpose: Evaluate the effect of payday periods on customer spending behavior.

```
SELECT
    CASE
        WHEN is_payday_period = 1 THEN 'Payday Period'
        WHEN is_payday_period = 0 THEN 'Non-Payday Period'
    END AS payday_status,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY is_payday_period;
```

Finding: Sales during payday periods are slightly higher than during non-payday periods, indicating increased purchasing power following salary disbursement.

6.9 Store Performance Analysis

Purpose: Identify the stores with the lowest sales performance (inefficient stores) based on sales and customer traffic.

A. Store Efficiency Correlation Analysis

```
SELECT
    store_avg_customer,
    store_avg_sales
FROM regresi;
```

B. Top 5 Inefficiency Stores by Average Sales

```
SELECT
    "Store" AS store_id,
    AVG("Customers_id") AS avg_customers,
    AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "Store"
ORDER BY avg_sales DESC
LIMIT 5;
```

C. Top 5 Inefficiency Stores by Average Customers

```
SELECT
  "Store" AS store_id,
  AVG("Customers_id") AS avg_customers,
  AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "Store"
ORDER BY avg_customers DESC
LIMIT 5;
```

Finding: Stores 262, 562, and 1,114 appear on both charts. Based on this data, these three stores are experiencing inefficiencies on both key metrics: they are listed among the stores with inefficient average sales as well as inefficient average customer counts.

7. Data Visualization

7.1 Metabase — Exploratory Dashboard

We have created an exploratory dashboard in Metabase to monitor day-to-day performance. This dashboard combines KPI cards (total revenue, total customers, and store performance) with a revenue bar chart by category and a monthly trend line.



Figure 7.1 — Metabase exploratory dashboard (KPI cards, Promotional activities, monthly trend, Store Inefficiency)

Interpretation: The dashboard shows that the business generated approximately €3.6 billion in total sales from 388 million customers across 1,115 stores, with an average basket size of €9.2. Although yearly sales declined slightly from 2014 to 2015, customer volume remained relatively stable, indicating consistent demand. Monthly sales and customer trends reveal seasonal fluctuations, with several peak periods throughout the year. Promotional activities had a strong positive impact on performance, generating significantly higher average sales than non-promotional periods. Sales were generally stronger during weekdays, while payday periods, Harbolnas events, and school holidays contributed to moderate increases in average sales, suggesting that special events and consumer purchasing cycles influence spending behavior. The store efficiency analysis indicates that stores with higher customer traffic tend to achieve higher sales, supported by a positive correlation between average customers and average sales. Overall, the findings suggest that customer traffic, promotions, seasonal factors, and event-driven campaigns are key drivers of revenue and store performance.

7.2 Power BI — Executive Dashboard

The final executive dashboard was built in Power BI, providing an overview of sales performance, customer traffic, promotional effectiveness, and payday-related purchasing behavior.

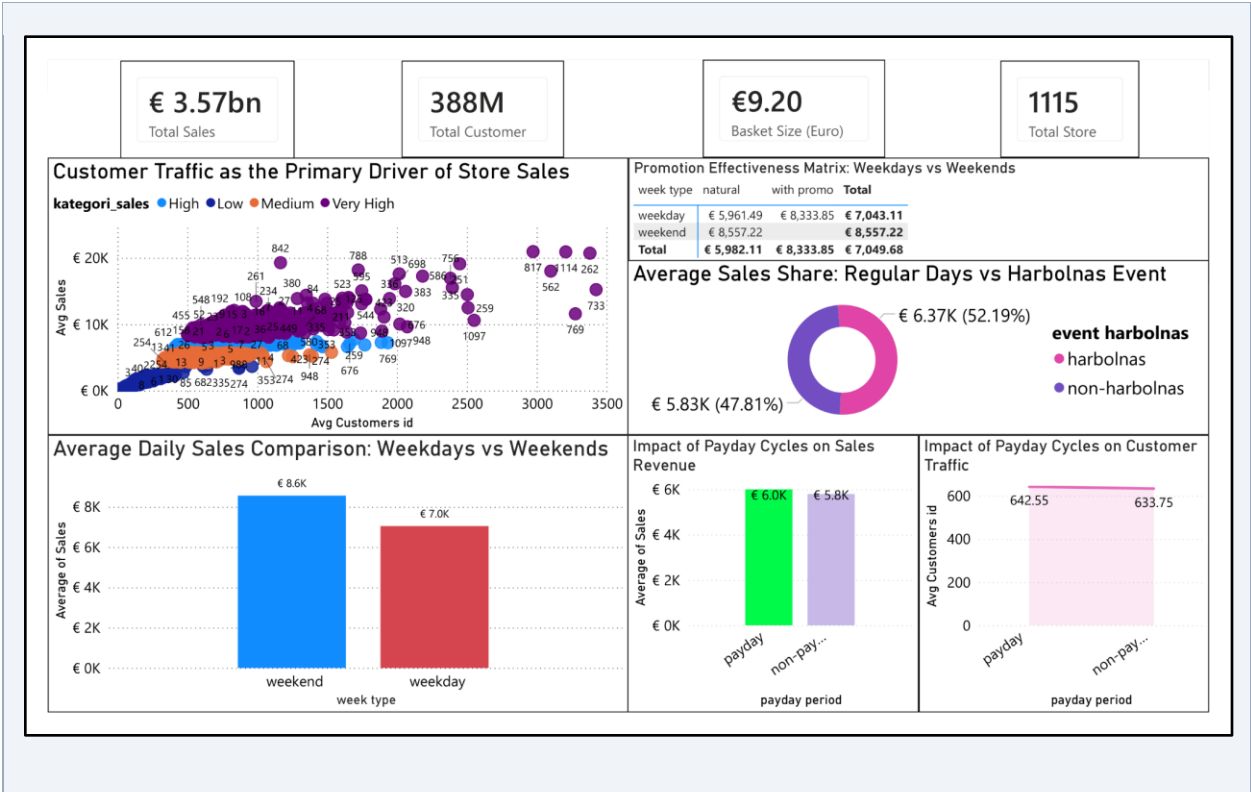


Figure 7.2 — Power BI executive dashboard

Interpretation: The executive dashboard shows that customer traffic is positively associated with store sales, as stores with higher customer volumes generally achieve higher sales performance. Promotional activities significantly improve sales, with promoted periods generating higher average sales than non-promoted periods. Harbolnas events also contribute positively to revenue, accounting for a slightly larger share of average sales compared to regular days. In addition, sales performance is stronger during weekends than weekdays, indicating higher customer spending during leisure periods. Payday cycles have a modest positive impact on sales, with average sales increasing during payday periods while customer traffic remains relatively stable. Overall, the findings suggest that promotions, special events, weekends, and payday periods play an important role in driving sales growth and enhancing store performance.

7.3 Supporting Chart — Revenue by Category sales

The bar chart below isolates revenue by revenue by sales category, which is the single most important driver in the analysis.

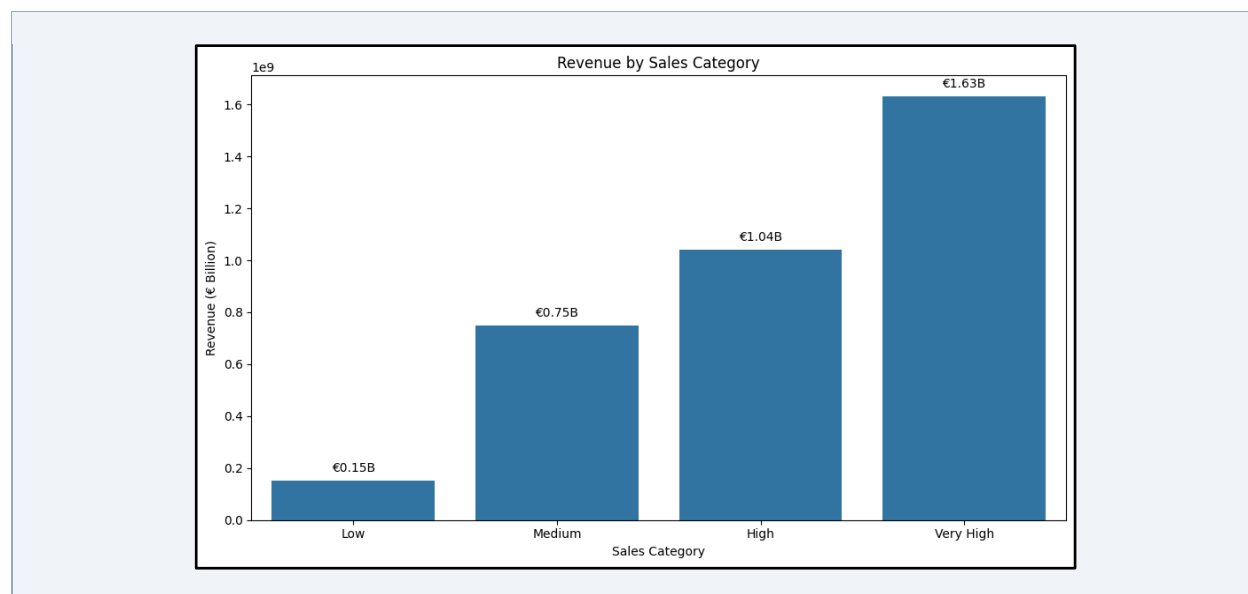


Figure 7.3 — Revenue by category sales

Interpretation: Revenue is heavily concentrated in the Very High and High sales categories, while the Low and Medium categories contribute a smaller share of total revenue. This indicates that a relatively small proportion of high-value transactions generates a significant portion of overall sales. Therefore, maintaining customer segments and business conditions that consistently produce High and Very High sales transactions should be a strategic priority. At the same time, opportunities exist to increase revenue by encouraging customers in the Low and Medium categories to move into higher spending segments through targeted promotions, cross-selling, and loyalty programs.

7.4 Streamlit— Sales Prediction Application

The final application was deployed using Streamlit. It enables users to enter relevant store and sales-related information and instantly obtain sales forecasts generated by the trained regression model. The web-based interface makes predictive analytics accessible to business users and supports informed decision-making.

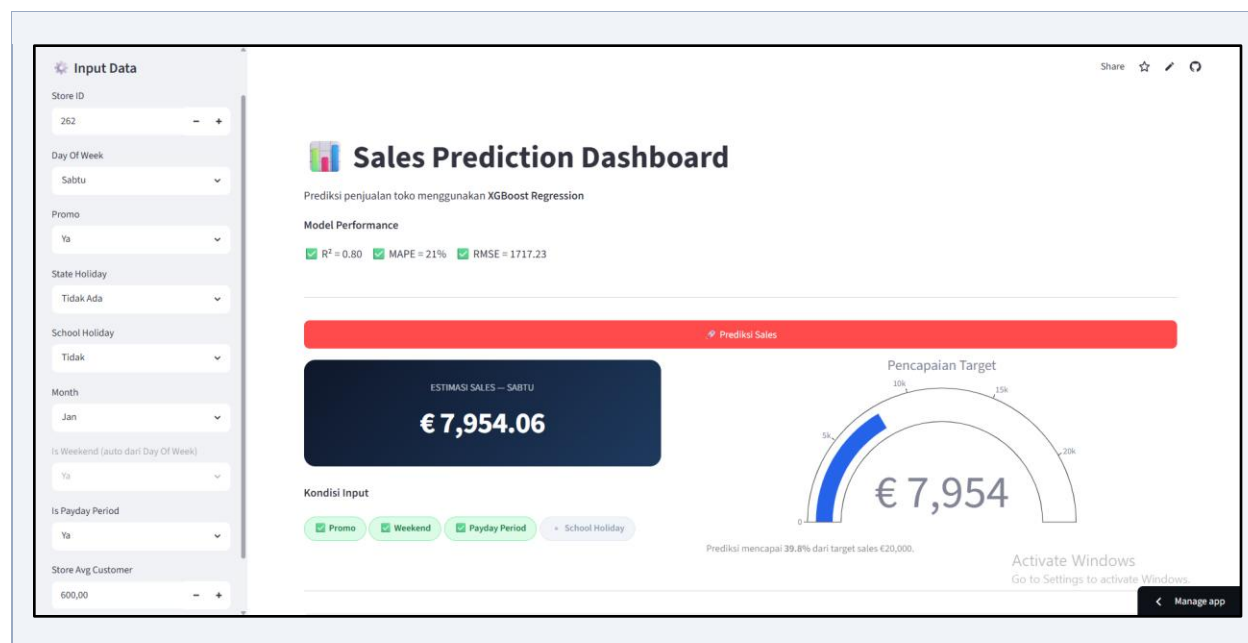


Figure 7.4 — Deploy Model

Interpretation: The application provides real-time sales predictions generated by the trained XGBoost regression model. By entering relevant store-related features, users can estimate expected sales under different scenarios, supporting data-driven planning, resource allocation, and decision-making.

8. Key Findings & Insights

- A. **Sales** and **Customer Trends**
Sales showed a declining trend alongside a decrease in customer numbers. This indicates that lower customer traffic was one of the main factors affecting the company's sales performance.
- B. **Monthly Sales Fluctuation**
Sales fluctuated throughout the observation period, with several months experiencing notable increases and others showing declines. This pattern suggests that sales were influenced by seasonal and time-related factors.
- C. **Promotion Effectiveness**
Promotional periods generated approximately **€1.9 billion** in total sales, exceeding the **€1.7 billion** generated during non-promotional periods. This suggests that promotions were effective in attracting transactions and increasing customer spending.
- D. **Payday-Period Sales**
Sales during payday periods were slightly higher than during non-payday periods, with average sales reaching approximately €6.0k compared to €5.8k outside the payday window. This suggests that customers tend to have greater purchasing power after receiving their salaries, leading to a modest increase in spending.

- E.

Weekday and Weekend

Performance

Sales performance was significantly stronger on weekdays than on weekends. Average sales on weekdays reached approximately €6.8k, while weekend sales averaged only €217. This indicates that customer transactions were heavily concentrated during weekdays, making them the primary driver of store revenue.
- F.

School-Holiday

Performance

School holiday periods generated higher average sales than non-school-holiday periods. Average sales increased from approximately €5.7k during regular periods to €6.6k during school holidays, suggesting that customers were more likely to shop and spend during holiday seasons.
- G.

Harbolnas

Event

Performance

Harbolnas events positively impacted sales performance, with average sales reaching approximately €6.4k compared to €5.8k on non-Harbolnas days. This indicates that the event successfully stimulated customer spending and contributed to stronger sales performance.
- H.

Relationship

Between

Customers

and

Sales

The analysis identified a strong positive relationship between customer traffic and sales. Stores categorized as **Very High Sales** were consistently associated with daily customer traffic of approximately 1,000–3,500 customers. This confirms that customer volume was a major driver of sales performance.
- I.

Underperforming

Stores

Stores **262, 562, and 1,114** appeared in both the low-sales and low-customer performance analyses. These stores were therefore identified as inefficient in the two main performance indicators: average sales and average customer traffic.
- J.

Regression

Model

Performance

Among the five models evaluated, **XGBoost Regression** produced the best predictive performance, achieving an **R² of 0.80, MAPE of 21.00%, MAE of 1,159.25, and RMSE of 1,717.23**. The model explained approximately 80% of the variation in sales and was selected for the sales-prediction dashboard.

9. Conclusion & Recommendations

9.1 Conclusion

Based on these findings, we conclude that the dataset for the sales regression:

- A. The company’s sales showed a fluctuating and declining trend alongside a decrease in customer numbers. Customer growth is therefore a key driver of improved business performance.
- B. Promotional activities proved highly effective. Payday periods and Harbolnas events contributed significantly to sales, indicating that purchasing behavior is strongly influenced by timing and promotions.
- C. Store efficiency varied considerably, with several stores consistently recording low customer traffic and sales. Optimizing customer acquisition strategies and evaluating store operations are therefore essential.
- D. A regression model was successfully developed to predict the company’s daily sales. Based on the evaluation results, XGBoost was the best-performing model, achieving an R² of 0.80. It was

integrated into an interactive dashboard to support more accurate operational planning and future sales-target setting.

9.2 Recommendations

Based on these findings, we recommend that the company:

A. Improve Customer Acquisition and Retention

The decline in customer numbers is consistent with the decrease in sales. Therefore, the company should strengthen its customer loyalty programs, implement more targeted promotions, and develop strategies to retain existing customers.

B. Optimize Promotions During High-Potential Periods

The analysis shows that promotions, payday periods, and Harbolnas contribute to increased sales. Promotional activities should therefore be focused on these periods to achieve better results.

C. Evaluate Underperforming Stores

Several stores recorded lower sales and efficiency than others. The company should evaluate these stores to identify the underlying causes and implement appropriate improvement strategies.

D. Use Sales Patterns for Business Planning

Fluctuating monthly sales patterns can serve as a basis for business planning, enabling the company to make better decisions based on historical data.

9.3 Limitations & Future Work

The dataset is fictional and covers only January 2014 to July 2015, meaning that 2015 does not represent a complete year. It also excludes product, cost, inventory, location, and competitor information, so profit and other external influences could not be evaluated. Future work could incorporate these variables, use multiple complete years to confirm seasonal patterns, and develop a separate customer-traffic model to improve future sales predictions.

10. Appendix

Appendix A — Full Python Script For Data Cleaning and Model Deployment

```
project4_regresi.py
# =====
# Import Data
# =====

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

df = pd.read_excel(
    "https://raw.githubusercontent.com/dzulfi02/Project4/master/regresi.xlsx")
df

# =====
# Read Data
```

```

# =====

df.info()
df.describe()

# =====
# Cleaning Data
# =====

df.isnull().sum()
df.duplicated().sum()
pd.set_option('display.max_columns', None)

# =====
# Feature Engineering
# =====

df['year'] = df['Date'].dt.year
df['month'] = df['Date'].dt.month
df['day_of_month'] = df['Date'].dt.day
df['week_of_year'] = df['Date'].dt.isocalendar().week.astype(int)
df['quarter'] = df['Date'].dt.quarter
df['is_weekend'] = df['Date'].dt.dayofweek.isin([5, 6]).astype(int)
df['is_payday_period'] = np.where(
    (df['day_of_month'] >= 25) | (df['day_of_month'] == 2),
    1, 0)
df['event_harbolnas'] = np.where(
    df['day_of_month'] == df['month'],
    1, 0)
df['customer_ratio'] = (
    df['Customers_id'] /
    df.groupby('Store')['Customers_id'].transform('mean'))
df['kategori_sales'] = pd.qcut(
    df['Sales'],
    q=4,
    labels=['Low', 'Medium', 'High', 'Very High'])
df['store_avg_customer'] = (
    df.groupby('Store')['Customers_id'].transform('mean'))
df['store_avg_sales'] = (
    df.groupby('Store')['Sales'].transform('mean'))

# =====
# Download Dataset
# =====

df.to_excel('regresi_final.xlsx', index=False)

# =====
# Correlation Analysis
# =====

df_num = df.select_dtypes(include=['int64', 'float64'])
corr = df_num.corr()

plt.figure(figsize=(12,8))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap")
plt.show()

# =====

```

```

# Regression Modeling
# =====

import statsmodels.api as sm
import warnings
warnings.filterwarnings("ignore")

from sklearn.linear_model import (
    LinearRegression, Ridge, Lasso)
from sklearn.svm import SVR
from xgboost import XGBRegressor
from sklearn.ensemble import (
    AdaBoostRegressor, GradientBoostingRegressor)
from sklearn.preprocessing import RobustScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    mean_absolute_percentage_error,
    r2_score)
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

# =====
# Split Data
# =====

X = df.drop(['Sales', 'Date'], axis=1)
y = df['Sales']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42)

# =====
# Ordinal Encoding
# =====

mapping = {
    'Low': 1,
    'Medium': 2,
    'High': 3,
    'Very High': 4}

X_train['kategori_sales'] = X_train['kategori_sales'].map(mapping)
X_test['kategori_sales'] = X_test['kategori_sales'].map(mapping)

# =====
# One Hot Encoding
# =====

X_train = pd.get_dummies(X_train, columns=['StateHoliday'], drop_first=True, dtype=int)
X_test = pd.get_dummies(X_test, columns=['StateHoliday'], drop_first=True, dtype=int)

# =====
# Feature Alignment
# =====

train_cols = X_train.columns

```



```
test_cols = X_test.columns

missing_in_test = set(train_cols) - set(test_cols)
for c in missing_in_test: X_test[c] = 0

missing_in_train = set(test_cols) - set(train_cols)
for c in missing_in_train: X_train[c] = 0

X_test = X_test[train_cols]

# =====
# Feature Selection
# =====

drop_cols = [
    'Customers_id',
    'year',
    'Open',
    'kategori_sales',
    'quarter',
    'event_harbolnas',
    'store_avg_sales',
    'customer_ratio']

X_train = X_train.drop(columns=drop_cols, errors='ignore')
X_test = X_test.drop(columns=drop_cols, errors='ignore')

# =====
# Feature Scaling
# =====

rs = RobustScaler()
X_train_transformed = rs.fit_transform(X_train)
X_test_transformed = rs.transform(X_test)

# =====
# Models
# =====

lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)

lasso = Lasso(alpha=1.0)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)

ada = AdaBoostRegressor(n_estimators=100, random_state=42)
ada.fit(X_train, y_train)
y_pred_ada = ada.predict(X_test)

xgb = XGBRegressor(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42,
```

```

    enable_categorical=True)

xgb.fit(X_train, y_train)
y_pred_xgb = xgb.predict(X_test)

# =====
# Model Evaluation
# =====

result = []
mask = y_test > 0

models = {
    'Linear Regression': y_pred_lr,
    'Ridge Regression': y_pred_ridge,
    'Lasso Regression': y_pred_lasso,
    'Adaboost Regression': y_pred_ada,
    'XGBoost Regression': y_pred_xgb}

for name, y_pred in models.items():

    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    mape = mean_absolute_percentage_error(
        y_test[mask], y_pred[mask]) * 100
    r2 = r2_score(y_test, y_pred)

    result.append([name, mae, rmse, mape, r2])

comparison_df = pd.DataFrame(
    result,
    columns=['Model', 'MAE', 'RMSE', 'MAPE (%)', 'R²']).round(2)

# =====
# Feature Importance
# =====

importance = pd.DataFrame({
    'Feature': X_train.columns,
    'Importance': xgb.feature_importances_
}).sort_values('Importance', ascending=False)

importance.head(10)

plt.figure(figsize=(10, 6))

ax = sns.barplot(
    data=importance.head(10),
    x='Importance',
    y='Feature')

ax.bar_label(ax.containers[0], fmt='%.3f', padding=5)

plt.title('Top 10 Feature Importance')
plt.xlim(0, importance['Importance'].max() * 1.1)
plt.show()

```

Appendix B — Full SQL Script for Finding Insights

project4_regresi.sql

```
-- Q1: Yearly Sales and Customer Trend
SELECT
    "year" AS year,
    COUNT(DISTINCT "Customers_id") AS customers,
    SUM("Sales") AS total_sales
FROM regresi
GROUP BY "year";

-- Q2: Monthly Sales Trend
SELECT
    DATE_TRUNC('month', "Date") AS month,
    SUM("Sales") AS total_sales
FROM regresi
GROUP BY DATE_TRUNC('month', "Date")
ORDER BY month;

-- Q3: Monthly Customer Trend
SELECT
    DATE_TRUNC('month', "Date") AS month,
    COUNT(DISTINCT "Customers_id") AS customers
FROM regresi
GROUP BY DATE_TRUNC('month', "Date")
ORDER BY month;

-- Q4: Average Sales by Day of Week and Promotion
SELECT
    CASE
        WHEN "DayOfWeek" = 1 THEN 'Monday'
        WHEN "DayOfWeek" = 2 THEN 'Tuesday'
        WHEN "DayOfWeek" = 3 THEN 'Wednesday'
        WHEN "DayOfWeek" = 4 THEN 'Thursday'
        WHEN "DayOfWeek" = 5 THEN 'Friday'
        WHEN "DayOfWeek" = 6 THEN 'Saturday'
        WHEN "DayOfWeek" = 7 THEN 'Sunday'
    END AS day_name,
    AVG(CASE WHEN "Promo" = 0 THEN "Sales" END) AS no_promo,
    AVG(CASE WHEN "Promo" = 1 THEN "Sales" END) AS promo
FROM regresi
WHERE "Open" = 1
GROUP BY "DayOfWeek"
ORDER BY "DayOfWeek";

-- Q5: Average Customers by Day of Week
SELECT
    CASE
        WHEN "DayOfWeek" = 1 THEN 'Monday'
        WHEN "DayOfWeek" = 2 THEN 'Tuesday'
        WHEN "DayOfWeek" = 3 THEN 'Wednesday'
        WHEN "DayOfWeek" = 4 THEN 'Thursday'
        WHEN "DayOfWeek" = 5 THEN 'Friday'
        WHEN "DayOfWeek" = 6 THEN 'Saturday'
        WHEN "DayOfWeek" = 7 THEN 'Sunday'
    END AS day_name,
    AVG("Customers_id") AS avg_customers
FROM regresi
WHERE "Open" = 1
GROUP BY "DayOfWeek"
ORDER BY "DayOfWeek";
```

```
-- Q6: Average Sales by Day of Week
SELECT
  CASE
    WHEN "DayOfWeek" = 1 THEN 'Monday'
    WHEN "DayOfWeek" = 2 THEN 'Tuesday'
    WHEN "DayOfWeek" = 3 THEN 'Wednesday'
    WHEN "DayOfWeek" = 4 THEN 'Thursday'
    WHEN "DayOfWeek" = 5 THEN 'Friday'
    WHEN "DayOfWeek" = 6 THEN 'Saturday'
    WHEN "DayOfWeek" = 7 THEN 'Sunday'
  END AS day_name,
  AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "DayOfWeek"
ORDER BY avg_sales DESC;

-- Q7: Weekend vs. Weekday Sales Comparison
SELECT
  CASE
    WHEN is_weekend = 1 THEN 'Weekend'
    WHEN is_weekend = 0 THEN 'Weekday'
  END AS period_type,
  SUM("Sales") AS total_sales,
  AVG("Sales") AS avg_sales,
  AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY is_weekend;

-- Q8: Harbolnas Event Sales Comparison
SELECT
  CASE
    WHEN event_harbolnas = 1 THEN 'Harbolnas Event'
    WHEN event_harbolnas = 0 THEN 'Non-Harbolnas'
  END AS event_status,
  SUM("Sales") AS total_sales,
  AVG("Sales") AS avg_sales,
  AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY event_harbolnas;

-- Q9: Promotion vs. Non-Promotion Sales Comparison
SELECT
  CASE
    WHEN "Promo" = 1 THEN 'Promotion'
    WHEN "Promo" = 0 THEN 'No Promotion'
  END AS promotion_status,
  SUM("Sales") AS total_sales,
  AVG("Sales") AS avg_sales,
  AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY "Promo";

-- Q10: School Holiday Sales Comparison
SELECT
  CASE
    WHEN "SchoolHoliday" = 1 THEN 'School Holiday'
    WHEN "SchoolHoliday" = 0 THEN 'Non-School Holiday'
  END AS holiday_status,
  SUM("Sales") AS total_sales,
```

```

        AVG("Sales") AS avg_sales,
        AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY "SchoolHoliday";

-- Q11: Payday Period Sales Comparison
SELECT
    CASE
        WHEN is_payday_period = 1 THEN 'Payday Period'
        WHEN is_payday_period = 0 THEN 'Non-Payday Period'
    END AS payday_status,
    SUM("Sales") AS total_sales,
    AVG("Sales") AS avg_sales,
    AVG("Customers_id") AS avg_customers
FROM regresi
GROUP BY is_payday_period;

-- Q12: Top 5 Stores by Average Sales
SELECT
    "Store" AS store_id,
    AVG("Customers_id") AS avg_customers,
    AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "Store"
ORDER BY avg_sales DESC
LIMIT 5;

-- Q13: Top 5 Stores by Average Customers
SELECT
    "Store" AS store_id,
    AVG("Customers_id") AS avg_customers,
    AVG("Sales") AS avg_sales
FROM regresi
GROUP BY "Store"
ORDER BY avg_customers DESC
LIMIT 5;

-- Q14: Store Efficiency Correlation Analysis
SELECT
    store_avg_customer,
    store_avg_sales
FROM regresi;

```

Appendix C — Python Script for Deployment

```

app.py
import streamlit as st
import joblib
import pandas as pd
import plotly.graph_objects as go

# =====
# PAGE CONFIG
# =====

st.set_page_config(

```

```

    page_title="Sales Prediction Dashboard",
    page_icon="📊",
    layout="wide")

# =====
# CUSTOM CSS
# =====

st.markdown("""
<style>
#MainMenu, footer {
    visibility: hidden;}

.chip {
    display: inline-block;
    padding: 6px 14px;
    border-radius: 20px;
    font-size: 0.82rem;
    font-weight: 600;
    margin: 4px 4px 0 0;}

.chip-on {
    background: #dcfce7;
    color: #15803d;
    border: 1px solid #86efac;}

.chip-off {
    background: #f1f5f9;
    color: #94a3b8;
    border: 1px solid #e2e8f0;}

.result-card {
    background: linear-gradient(135deg, #0f172a 0%, #1e3a5f 100%);
    border-radius: 16px;
    padding: 28px 32px;
    color: white;
    text-align: center;}

.result-card .label {
    font-size: 0.85rem;
    opacity: 0.7;
    margin-bottom: 6px;}

.result-card .value {
    font-size: 2.6rem;
    font-weight: 700;
    letter-spacing: -1px;}
</style>
""", unsafe_allow_html=True)

# =====
# LOAD MODEL
# =====

@st.cache_resource
def load_model():
    return joblib.load("model_xgb.pkl")

try:
    loaded = load_model()

```

```

        model = loaded["model"] if isinstance(loaded, dict) and "model" in loaded else loaded

except Exception as e:
    st.error(f"❌ Gagal memuat model: {e}")
    st.info("Pastikan file 'model_xgb.pkl' berada di folder yang sama dengan app.py")
    st.stop()

# =====
# HEADER
# =====

st.title("📊 Sales Prediction Dashboard")

st.markdown("""
Prediksi penjualan toko menggunakan **XGBoost Regression**

### Model Performance
☒ **R2 = 0.80**
☒ **MAPE = 21%**
☒ **RMSE = 1717.23**""")

st.divider()

# =====
# SIDEBAR INPUT
# =====

st.sidebar.header("⚙️ Input Data")

DAY_NAMES = [
    "Senin", "Selasa", "Rabu", "Kamis", "Jumat", "Sabtu", "Minggu"]

MONTH_NAMES = [
    "Jan", "Feb", "Mar", "Apr", "Mei", "Jun", "Jul", "Agu", "Sep", "Okt", "Nov", "Des"]

store = st.sidebar.number_input(
    "Store ID",
    min_value=1,
    max_value=1115,
    value=1)

day_of_week = st.sidebar.selectbox(
    "Day Of Week",
    [1, 2, 3, 4, 5, 6, 7],
    format_func=lambda x: DAY_NAMES[x - 1])

promo = st.sidebar.selectbox(
    "Promo",
    [1, 0],
    format_func=lambda x: "Ya" if x else "Tidak")

state_hol = st.sidebar.selectbox(
    "State Holiday",
    ["0", "a", "b", "c"],
    format_func=lambda x: {
        "0": "Tidak Ada",
        "a": "Public Holiday",
        "b": "Easter",
        "c": "Christmas"
    }[x])

```

```

    }[x])

school_hol = st.sidebar.selectbox(
    "School Holiday",
    [0, 1],
    format_func=lambda x: "Ya" if x else "Tidak")

month = st.sidebar.selectbox(
    "Month",
    list(range(1, 13)),
    format_func=lambda x: MONTH_NAMES[x - 1])

# Auto Weekend

is_weekend = 1 if day_of_week in [6, 7] else 0

st.sidebar.selectbox(
    "Is Weekend (Auto)",
    [is_weekend],
    format_func=lambda x: "Ya" if x else "Tidak",
    disabled=True)

is_payday_period = st.sidebar.selectbox(
    "Is Payday Period",
    [1, 0],
    format_func=lambda x: "Ya" if x else "Tidak")

store_avg_customer = st.sidebar.number_input(
    "Store Avg Customer",
    min_value=0.0,
    value=600.0,
    step=10.0)

# =====
# FEATURE ENGINEERING
# =====

today = pd.Timestamp.now()

day_of_month = int(today.day)
week_of_year = int(today.isocalendar().week)

state_hol_a = 1 if state_hol == "a" else 0
state_hol_b = 1 if state_hol == "b" else 0
state_hol_c = 1 if state_hol == "c" else 0

nama_hari = DAY_NAMES[day_of_week - 1]

# =====
# INPUT DATAFRAME
# =====

input_data = pd.DataFrame([
    "Store": store,
    "DayOfWeek": day_of_week,
    "Promo": promo,
    "SchoolHoliday": school_hol,
    "month": month,
    "day_of_month": day_of_month,
    "week_of_year": week_of_year,

```



```

        "is_weekend": is_weekend,
        "is_payday_period": is_payday_period,
        "store_avg_customer": store_avg_customer,
        "StateHoliday_a": state_hol_a,
        "StateHoliday_b": state_hol_b,
        "StateHoliday_c": state_hol_c]])

try: expected_cols = model.get_booster().feature_names

    if expected_cols is not None:
        input_data = input_data[expected_cols]

except Exception: pass

# =====
# PREDICTION
# =====

if st.button(
    "🔮 Prediksi Sales",
    use_container_width=True,
    type="primary"):

    try: pred = model.predict(input_data)[0]

    except Exception as e:
        st.error(f"❌ Gagal prediksi: {e}")
        st.info("Periksa kembali urutan kolom dan tipe data input.")
        st.stop()

    target = 20000.0
    achievement = min((pred / target) * 100, 100)

    col_left, col_right = st.columns([1, 1.3], gap="large")

    # =====
    # RESULT CARD
    # =====

    with col_left:
        st.markdown(f"""
        <div class="result-card">
            <div class="label">
                ESTIMASI SALES — {nama_hari.upper()}
            </div>
            <div class="value">
                € {pred:,.2f}
            </div>
        </div>
        """, unsafe_allow_html=True)

        st.markdown("<br>", unsafe_allow_html=True)
        st.markdown("***Kondisi Input**")

        chips = ""
        chips += f'<span class="chip {"chip-on" if promo else "chip-off"}">{"✅" if promo
else "❌"} Promo</span>'
        chips += f'<span class="chip {"chip-on" if is_weekend else "chip-off"}">{"✅" if
is_weekend else "❌"} Weekend</span>'

```

```

        chips += f'<span class="chip {"chip-on" if is_payday_period else "chip-off"}">{"☒ " if is_payday_period else "☐"} Payday Period</span>'
        chips += f'<span class="chip {"chip-on" if school_hol else "chip-off"}">{"☒ " if school_hol else "☐"} School Holiday</span>'

    st.markdown(chips, unsafe_allow_html=True)

    # =====
    # GAUGE CHART
    # =====

    with col_right:
        fig = go.Figure(
            go.Indicator(
                mode="gauge+number",
                value=pred,
                number={
                    "prefix": "€ ",
                    "valueformat": ",.0f"},
                title={"text": "Pencapaian Target"},
                gauge={
                    "axis": {"range": [0, target * 1.2]},
                    "bar": {"color": "#2563eb"}}))

        fig.update_layout(
            height=240,
            margin=dict(
                t=40,
                b=10,
                l=20,
                r=20))

        st.plotly_chart(
            fig,
            use_container_width=True)

        st.caption(
            f"Prediksi mencapai **{achievement:.1f}%** "
            f"dari target sales €{target:,.0f}.")

    st.divider()

    # =====
    # INPUT SUMMARY
    # =====

    with st.expander("📄 Ringkasan Input"):
        st.write(f"""
        **Store ID:** {store}
        **Day Of Week:** {nama_hari}
        **Day Of Month:** {day_of_month}
        **Month:** {month}
        **Week Of Year:** {week_of_year}
        **Promo:** {"Ya" if promo else "Tidak"}
        **Is Weekend:** {"Ya" if is_weekend else "Tidak"}
        **Is Payday Period:** {"Ya" if is_payday_period else "Tidak"}
        **School Holiday:** {"Ya" if school_hol else "Tidak"}
        **State Holiday:** {state_hol}
        **Store Avg Customer:** {store_avg_customer:,.0f}""")

```

```

        st.dataframe(
            input_data,
            use_container_width=True)

# =====
# DEFAULT VIEW
# =====

else:
    st.markdown("""
    <div style="
        border: 2px dashed #cbd5e1;
        border-radius: 16px;
        padding: 48px 32px;
        text-align: center;
        color: #94a3b8;
        margin-top: 8px;">
        <div style="font-size: 2.5rem; margin-bottom: 12px">
            📊
        </div>

        <div style="
            font-weight: 600;
            font-size: 1rem;
            margin-bottom: 6px;">
            Hasil prediksi akan muncul di sini
        </div>

        <div style="font-size: 0.83rem">
            Lengkapi input di sidebar, lalu klik
            <strong>Prediksi Sales</strong>
        </div>
    </div>""", unsafe_allow_html=True)

# =====
# FOOTER
# =====
st.divider()
st.caption("📊 Sales Prediction Dashboard | XGBoost Regression")

```

Appendix D — Team Contribution Table

Each member confirms the contributions recorded below by signing.

Member	Main Responsibilities	Contribution %	Signature
Dzulfi Khoiriyah Azzahra	Python data cleaning & preprocessing; SQL analysis & queries; Metabase ; Visualization; Documentation.	35%	
Pradika Mekanata	SQL analysis & queries; Methodology; Key Findings; Metabase & Power BI dashboards; Visualization.	30%	
Rangga Adi Nugraha	Python data cleaning & preprocessing; Modeling; Deployment; Documentation	35%	

Table 10.1 — Team contribution summary (total must equal 100%)

